Git & GitHub Masterclass: From Basics to Advanced Workflows

Master version control, collaboration, and AI-powered development with our comprehensive 10-module course covering Git fundamentals, GitHub automation, Copilot, VS Code essentials, and more.

Enroll Now | View Curriculum



Module 1: Git & GitHub Fundamentals



Understanding Version Control

Learn the core concepts of version control and why it's essential for modern development workflows.



Creating Repositories & Commits

Master the fundamentals of creating repositories and making meaningful commits to track your project history.



Team Branching Strategies

Discover effective branching strategies for team collaboration and learn the GitHub Flow methodology.



Hands-on Practice

Create your first repository and pull request in a guided hands-on exercise to solidify your understanding.

Git in 60 Seconds

Fast & Decentralized

Git is a decentralized version control system designed for speed and efficiency, making it an indispensable tool for developers.

Essential Commands

Master key commands like init, clone, add, commit, and push to effectively manage your codebase.

Snapshot Principle

Unlike other systems, Git uses snapshots instead of file comparisons, creating a complete picture of your project at each commit.

Local vs. Remote

Understand the relationship between local and remote branches and how they synchronize in a distributed workflow.

GitHub Essentials



Hosted Git Repositories

GitHub provides cloud-hosted Git repositories with powerful collaboration features that extend Git's capabilities.



Collaboration Tools

Leverage repositories, issues, pull requests, and wikis to streamline your team's development process.



Pricing Tiers

Compare free, Pro, and Enterprise plans to find the right fit for your individual or organizational needs.



Security Features

Implement branch protection rules and `CODEOWNERS` files to maintain code quality and security standards.

Branching & Merging Best Practices

1

Feature Branch Workflow

Create isolated branches for each feature or bug fix to maintain a clean and stable main branch.

Commit Message Conventions

Follow Conventional Commits standards to create clear, consistent, and meaningful commit messages.

Pull Request Reviews

Implement thorough code reviews and automated checks to ensure code quality before merging.

4

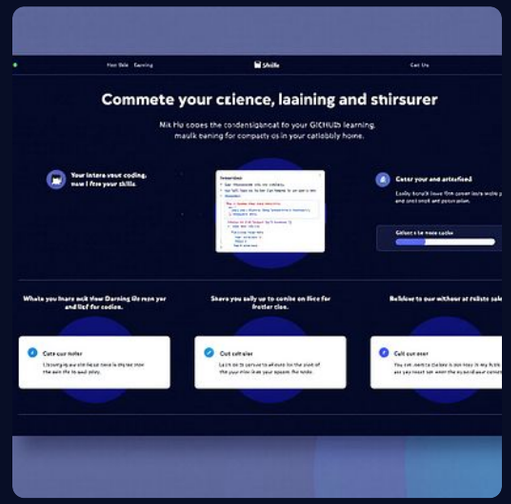
Conflict Resolution

Learn techniques for resolving merge conflicts using rebase or merge strategies effectively.

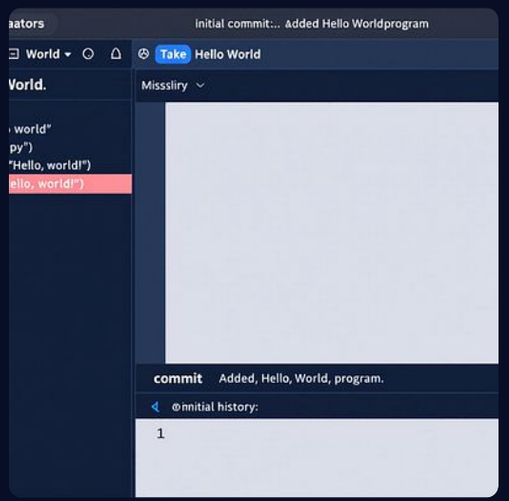
Module 1 Wrap-up & Resources





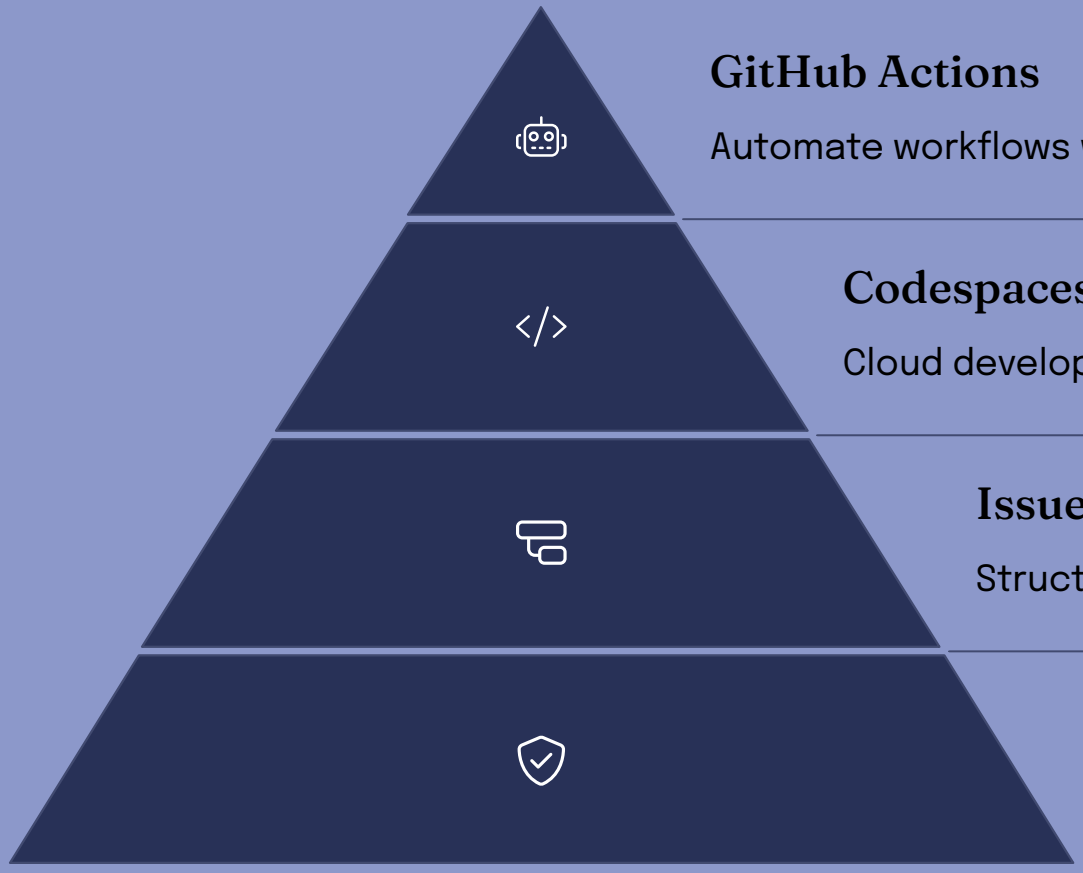






Solidify your Git knowledge with comprehensive resources including cheatsheets, the "Pro Git" e-book, GitHub Skills tutorials, and the interactive Learn Git Branching tool. Complete your learning by pushing your first Hello-World repository and testing your knowledge with a quick quiz.

Module 2: Collaboration & GitHub Automation



GitHub Actions

Automate workflows with powerful CI/CD capabilities

Codespaces & DevContainers

Cloud development environments for instant coding

Issue Workflows & Projects

Structured task management and tracking

Branch Policies & CI Checks

Code quality enforcement and protection

Module 2 focuses on advanced GitHub collaboration features and automation tools that streamline development workflows. You'll learn to implement GitHub Actions for continuous integration, set up cloud development environments with Codespaces, manage projects with structured issue workflows, and enforce code quality with branch policies.

GitHub Actions Fundamentals

YAML Workflow Syntax

Master the structure and syntax of workflow files that power GitHub Actions

Actions Marketplace

Leverage pre-built actions from the marketplace to accelerate development



Workflow Triggers

Configure events like push, pull_request, and cron schedules to start workflows

Runner Types

Choose between Ubuntu, Windows, and macOS runners for your workflow needs

Codespaces & DevContainers

< 10s

Startup Time

Launch cloud-based VS Code environments in seconds

100%

Consistency

Ensure identical development environments across team members

5GB

Persistent Storage

Default storage for settings and extensions

60h

Free Tier

Monthly quota for GitHub Pro users

GitHub Codespaces provides instant cloud development environments with VS Code in your browser. Configure your environment with .devcontainer.json to specify dependencies, extensions, and settings that persist between sessions. Be mindful of quota limits and costs when planning your Codespace usage.



Issues, Projects & Wiki

Structured Issue Management

Create custom issue templates with predefined sections and labels to standardize bug reports and feature requests. Implement automated issue responses using GitHub Actions to improve workflow efficiency.

- Template-driven issue creation
- Custom labels for categorization
- Automated responses and assignments

Agile Project Boards

Organize work with flexible project boards that support agile methodologies. Track progress visually and automate status updates based on issue events.

- Kanban and Scrum board templates
- Automated card movement
- Sprint planning integration

Documentation Options

Choose between README files for essential information and Wiki pages for comprehensive documentation. Establish clear "Definition of Done" criteria to maintain quality standards.

- Wiki vs. README best practices
- Markdown formatting guidelines
- Documentation maintenance workflow

Module 2 Wrap-up & Resources

Actions Starter Workflows

Explore pre-configured workflow templates for common CI/CD scenarios to jumpstart your automation journey.

Codespaces Billing Guide

Understand the cost structure and quota management for efficient Codespaces usage in your projects.

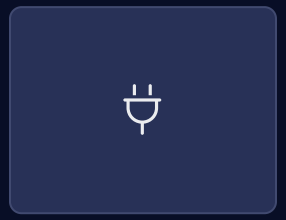
Project Board Templates

Leverage ready-made project board layouts designed for different development methodologies and team sizes.

Practical Exercise

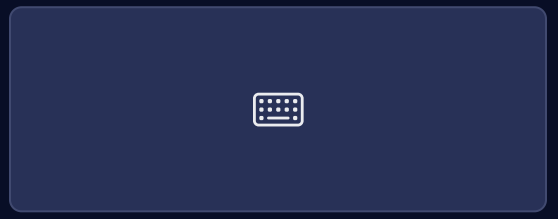
Apply your knowledge by implementing a continuous integration workflow for Python tests in your repository.

Module 3: Copilot Basics (VS Code)



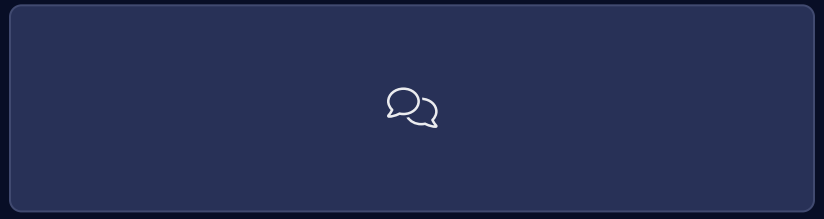
Installation & Activation

Set up GitHub Copilot in your development environment



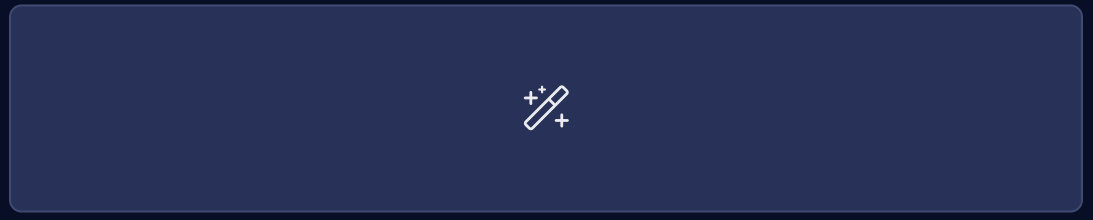
Inline Completion Mastery

Efficiently work with AI-suggested code completions



Chat Command Basics

Leverage Copilot Chat for code explanations and fixes



Prompt Design

Craft effective prompts for better suggestions

Module 3 introduces you to GitHub Copilot, an AI pair programmer that helps you write code faster and with fewer errors. You'll learn how to install and configure Copilot, master inline completions, use Copilot Chat effectively, and design prompts that generate high-quality code suggestions.

Copilot Installation & Setup

Extension Installation & Login Flow

> Install the GitHub Copilot extension from the VS Code marketplace and complete the authentication process by signing in with your GitHub account. Ensure you have an active Copilot subscription or access through your organization.

Configuration Settings

> Customize Copilot's behavior through settings like suggestion frequency, telemetry options, and language-specific preferences. You can also configure API keys for enterprise environments that use custom proxies.

Essential Shortcuts

> Master key shortcuts to boost productivity: Tab to accept suggestions, ⌘I to trigger inline suggestions, ⌘⏎ for additional options, and configure a "No suggestions" key to temporarily disable Copilot when needed.

Troubleshooting Common Issues

> Learn how to resolve common setup problems including authentication failures, suggestion delays, and compatibility issues with other extensions or environments.

Prompt-Crafting 101

1

Provide Context

Open relevant files and select code to give Copilot the necessary context for accurate suggestions.

Define Goals

Clearly specify your objective, preferred coding style, and target language in your prompts.

Use Negative Prompts

Explicitly state what to avoid in the generated code to improve relevance.

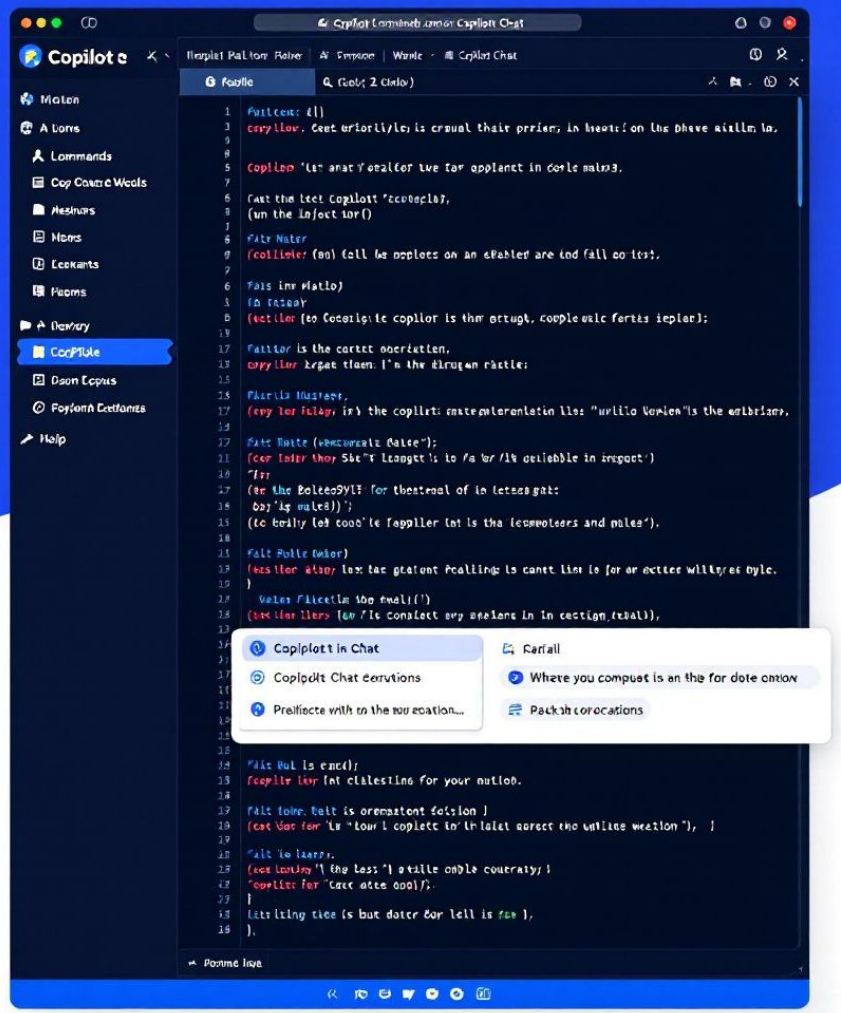
Multi-line Comments

Use detailed comment blocks to provide comprehensive instructions for complex tasks.



Copilot Chat ends

Whien your wo distirc mefessional VS white your hare af the onicw you conllot ance. cupt for ther in your fille insteratic plances and a fiffictional ments.



Copilot Chat Cheat-Sheet

Essential Commands

Master key commands like /explain to understand code, /fix-tests to repair failing tests, and /generate to create new code based on specifications.

Interface Options

Choose between inline chat for quick queries, panel view for context-specific help, or sidebar mode for ongoing conversations while coding.

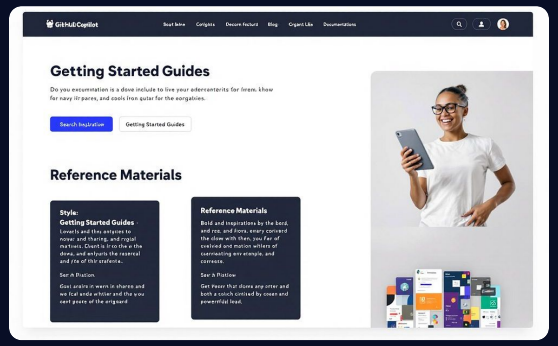
Managing Suggestions

Learn efficient workflows for accepting, modifying, or rejecting Copilot's suggestions to maintain code quality.

Keyboard Efficiency

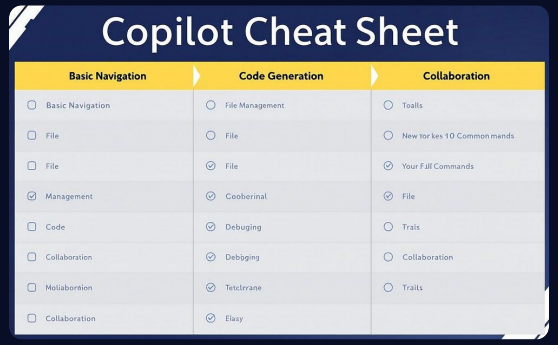
Use the ⇧⌘I shortcut to instantly start a chat session without interrupting your coding flow.

Module 3 Wrap-up & Resources



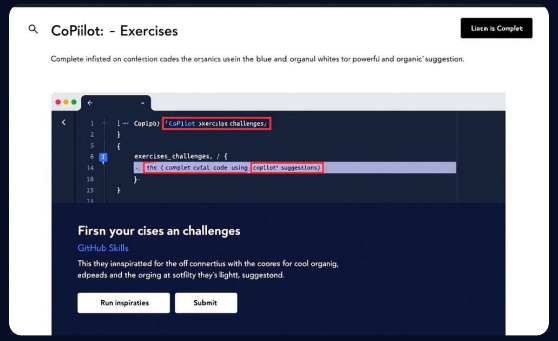
Official Documentation

Access comprehensive guides, tutorials, and reference materials in the GitHub Copilot documentation to deepen your understanding of AI-assisted coding.



Chat Command Cheat Sheet

Download and reference the complete Copilot Chat command reference to quickly find the right commands for your coding tasks.



Interactive Training

Complete the "Getting Started with Copilot" course on GitHub Skills to practice your prompt-crafting abilities with guided exercises.

Module 4: Copilot Agent-Mode & Workspace

Chat vs. Agent Differences

Understand how Agent-Mode extends beyond chat functionality to execute multi-step tasks autonomously.

Agent Workflow Planning

Learn to structure complex development tasks for optimal Agent execution and oversight.

Custom Agent Configuration

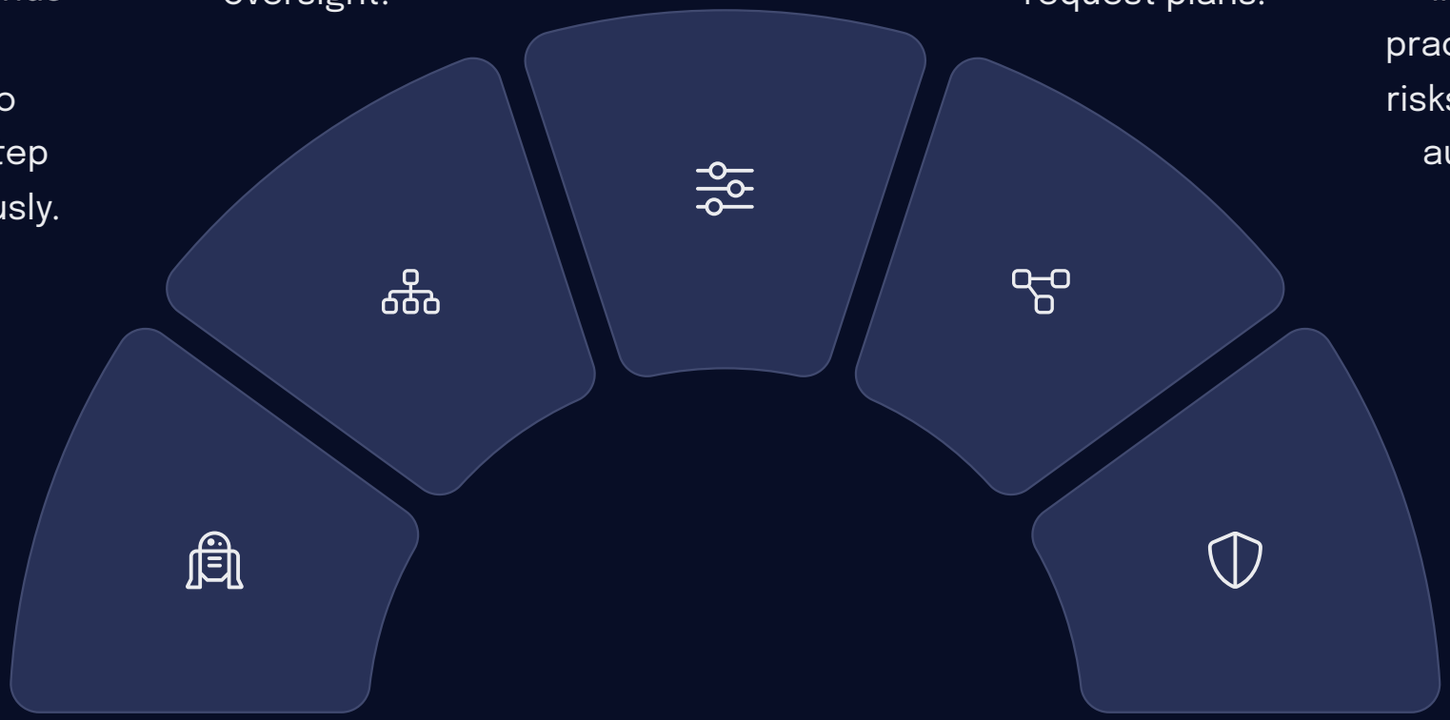
Configure agents with specific capabilities and permissions tailored to your project needs.

Workspace & PR Plans

Leverage Copilot Workspace to generate and execute comprehensive pull request plans.

Risk Management

Implement best practices to mitigate risks associated with automated code generation.



Agent-Mode Overview

Command & Planning

Initiate Agent-Mode with the /agent command followed by a detailed task description. The agent generates a comprehensive plan that you can review before execution begins.

- Preview generated plans before execution
- Modify plans to match your requirements
- Set constraints and boundaries for the agent

Execution Capabilities

Agents can perform complex operations across multiple files, run tests to verify changes, and execute terminal commands to complete development tasks.

- Edit multiple files in a single operation
- Run and fix tests automatically
- Execute terminal commands safely
- Implement complete features end-to-end

Control & Safety

Maintain control with options to undo changes or abort operations at any point in the execution process.

- Step-by-step execution with approval
- Undo specific changes when needed
- Abort the entire operation safely
- Review all changes before committing

Workspace & Session-Sync

Workspace Extension Setup

Install the Copilot Workspace extension in VS Code to enable seamless integration between browser-based planning and code implementation.

Browser to Editor Workflow

Generate comprehensive plans in the browser interface and transfer them directly to VS Code for execution with full context preservation.

Codespaces Integration

Leverage the tight integration between Workspace and Codespaces for a fully cloud-based AI-assisted development experience.

Collaborative Planning

Use comment threads within plans to collaborate with team members on implementation details before and during execution.

Custom Agents & Extensions

Agents API Preview

Explore the preview version of the Agents API that allows developers to create specialized AI assistants for specific domains and workflows.

YAML Manifest Structure

Learn to define agent capabilities through YAML manifests that specify available tools, required permissions, and operational constraints.

Domain-Specific Actions

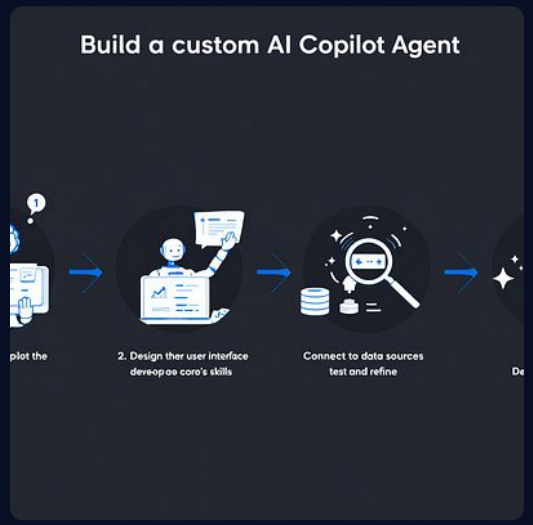
Create custom actions tailored to specific technologies, frameworks, or business domains to enhance productivity in specialized development contexts.

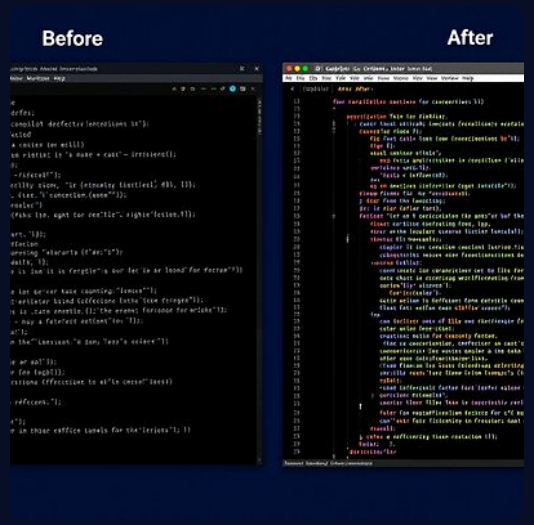
Security Considerations

Understand the security implications of agent permissions and implement appropriate scopes to protect sensitive resources and operations.

Module 4 Wrap-up & Resources



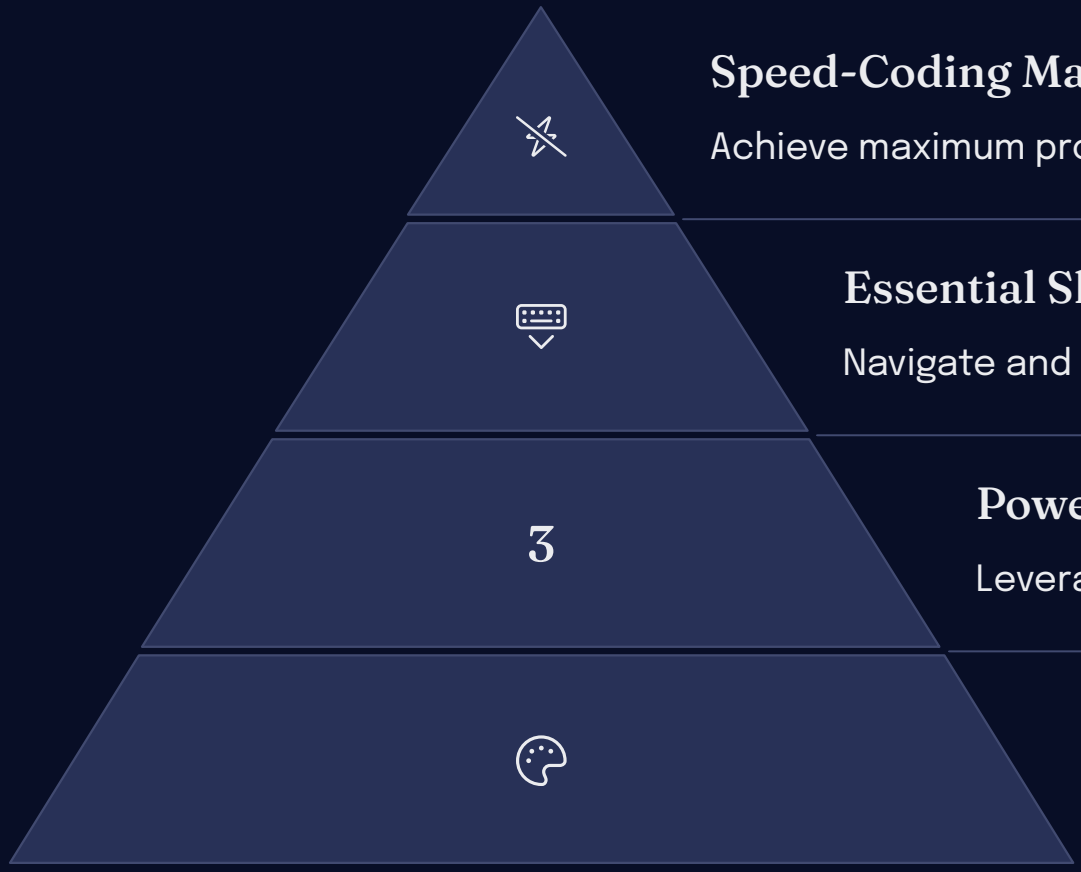






Deepen your understanding of Copilot's Agent-Mode with official resources including the introductory blog post and comprehensive documentation on building custom agents. Practice your skills with a guided mini-refactoring exercise, and always reference the security review checklist before running agent operations in production environments.

Module 5: VS Code Essentials



Speed-Coding Mastery

Achieve maximum productivity with VS Code

Essential Shortcuts

Navigate and edit code with keyboard efficiency

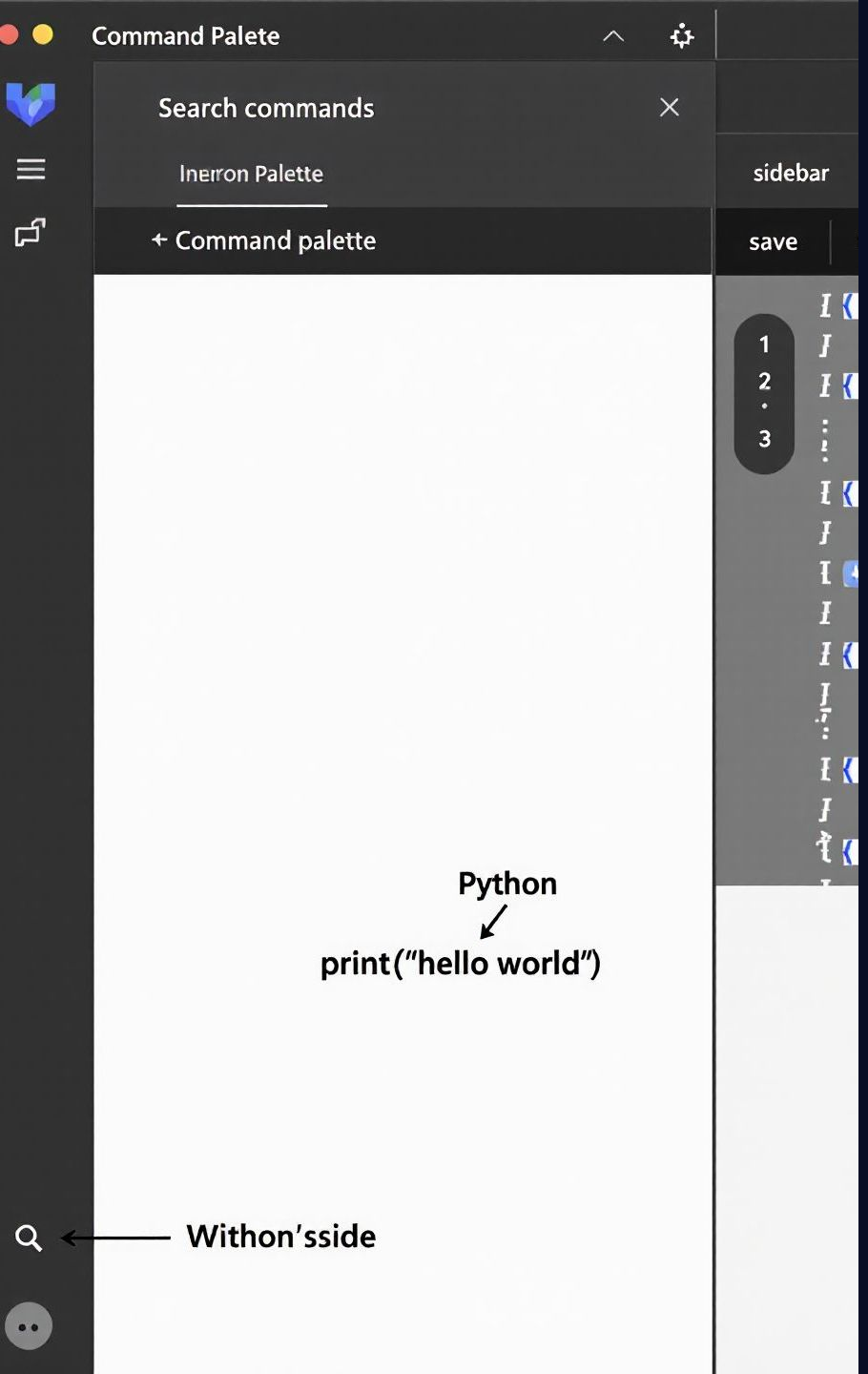
Power-User Features

Leverage advanced editing and navigation tools

Personalization

Customize your environment for optimal workflow

Module 5 focuses on mastering VS Code's interface and features to dramatically improve your coding efficiency. You'll learn essential keyboard shortcuts, powerful editing techniques like multi-cursor editing, personalization options including themes and settings sync, and explorer power-user tricks that will transform your development workflow.



UI Tour & Shortcuts

Command Palette (⌘⇧P)

Access all VS Code commands instantly through this powerful search interface. Simply type what you want to do in natural language to find the right command.

Interface Sections

Master the Side Bar for file navigation, Panel for output and terminals, and Status Bar for quick information and actions.

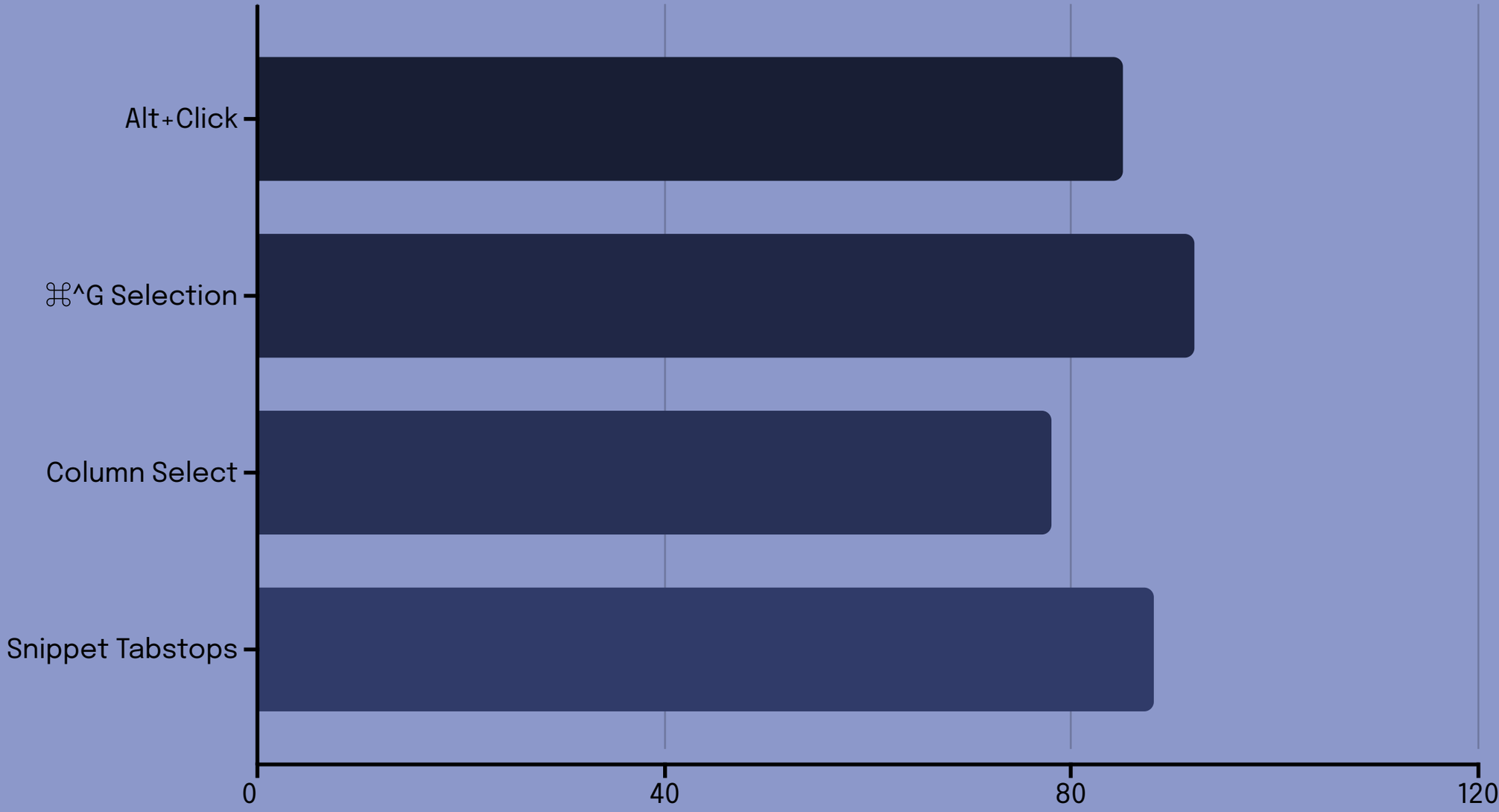
Quick Open (⌘P)

Navigate to any file in your workspace instantly by typing part of its name, bypassing folder hierarchy completely.

Navigation Aids

Use Breadcrumbs and Outline view to understand code structure and jump between sections efficiently.

Multi-Cursor Magic



Multi-cursor editing is one of VS Code's most powerful features. Place multiple cursors with Alt+Click (⌥Click on Mac) or select all occurrences of the current selection with ⌘⌥G. For vertical selections, use column select with Alt+Shift+drag (⌥⇧ drag on Mac). Combine with snippets using $1, $2 tabstops to transform your editing efficiency.

Settings Sync & Themes

GitHub-Powered Sync

Connect VS Code to your GitHub account to synchronize your development environment across multiple machines. This ensures consistent experience regardless of which computer you're using.

- One-click setup with GitHub login
- Automatic background synchronization
- Conflict resolution for setting changes

Synchronized Content

Settings Sync preserves your entire VS Code configuration including settings, keybindings, snippets, extensions, and UI state.

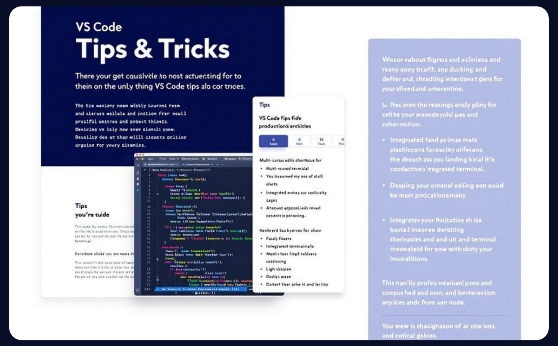
- User and workspace settings
- Custom keybindings and shortcuts
- Installed extensions and their settings
- UI state and layout preferences

Personalization Options

Customize your coding environment with trending themes, icon packs, and workspace-specific settings to optimize for different projects.

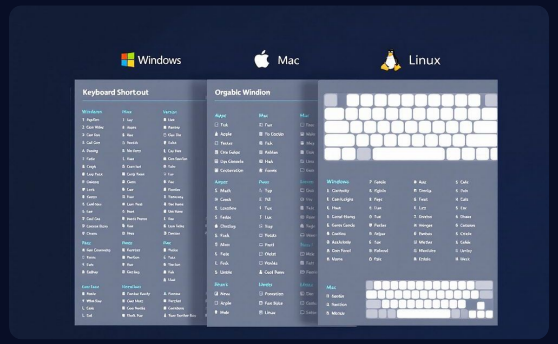
- Popular 2025 theme recommendations
- File icon themes for visual organization
- Project-specific setting overrides
- Font and editor customization

Module 5 Wrap-up & Resources



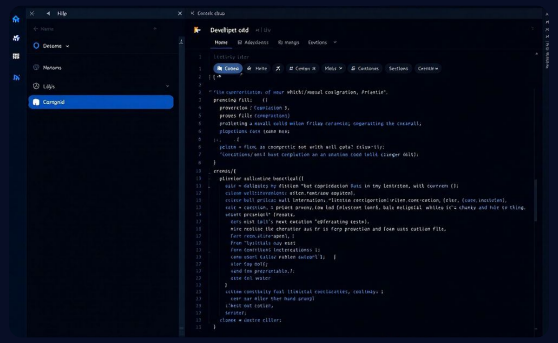
Tips & Tricks Guide

Access the official VS Code "Tips & Tricks" guide for a comprehensive collection of productivity techniques and hidden features that will accelerate your development workflow.



Keyboard Shortcut References

Download platform-specific cheat sheets for Windows, Mac, and Linux to master the keyboard shortcuts most relevant to your operating system.



Custom Snippet Exercise

Practice creating your own code snippets to automate repetitive coding patterns and further boost your productivity with personalized shortcuts.

Module 6: Terminal & Tasks

Integrated Terminal Mastery

Learn to leverage VS Code's built-in terminal for seamless command-line operations without leaving your editor.

2 Multiple Shell Profiles

Configure and switch between different shell environments optimized for specific development tasks.

Automated Task Configuration

Set up automated build, test, and deployment tasks to streamline your development workflow.

Environment & Secrets Management

Securely manage environment variables and sensitive information in your development process.

Terminal Deep-Dive

Quick Access (^`)

Open the integrated terminal instantly with the Control+backtick shortcut, keeping your hands on the keyboard for maximum efficiency.

Split Terminals (⌘\)

Run multiple terminal sessions side by side with the split pane feature, perfect for monitoring logs while executing commands.

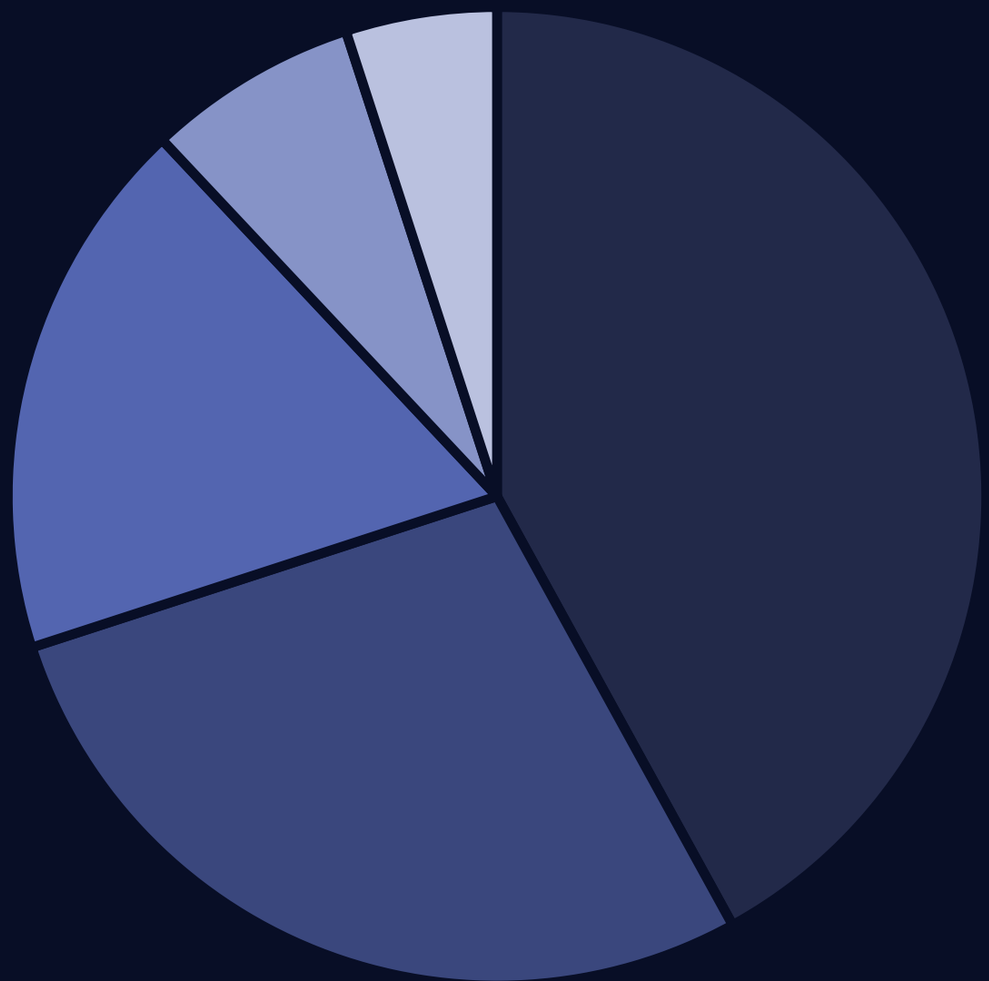
Enhanced Text Handling

Benefit from automatic link detection and powerful search capabilities (⌘F) that make working with terminal output more efficient.

Git Integration

Execute Git commands directly in the terminal while seeing visual indicators of changes in the editor and source control panel.

Shell Profiles & Environment Variables



Configure multiple shell profiles using the "terminal.integrated.profiles.*" setting to switch between different command-line environments. Set default shells per operating system and manage environment variables through .env files and the "terminal.integrated.env.*" settings. Securely store sensitive information using VS Code's built-in secret storage and configure proxy settings and SSH key pass-through for remote development.

Tasks Basics

1

Task Configuration Structure

Create a tasks.json file to define automated processes with labels, commands, and problem matchers.

Task Execution

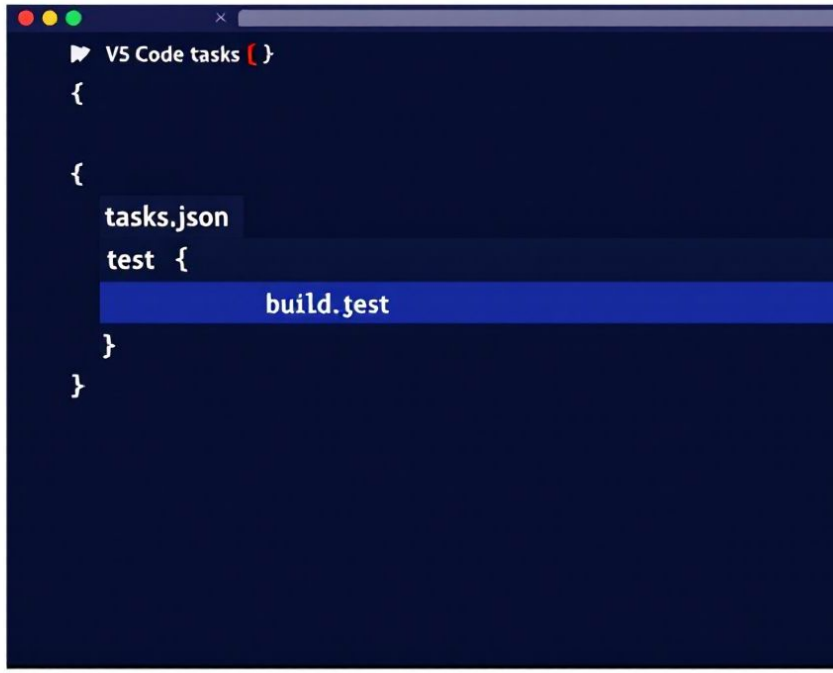
Run tasks manually or configure them to execute automatically before debugging or deployment.

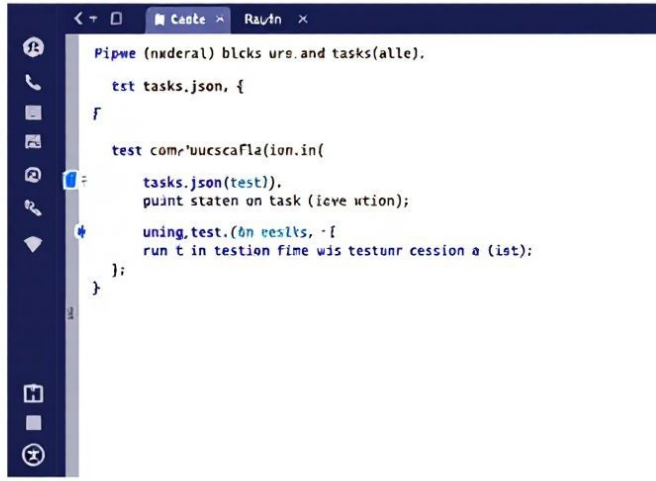
Pre-Launch Tasks

Set up build or preparation tasks that run automatically before launching a debug session.

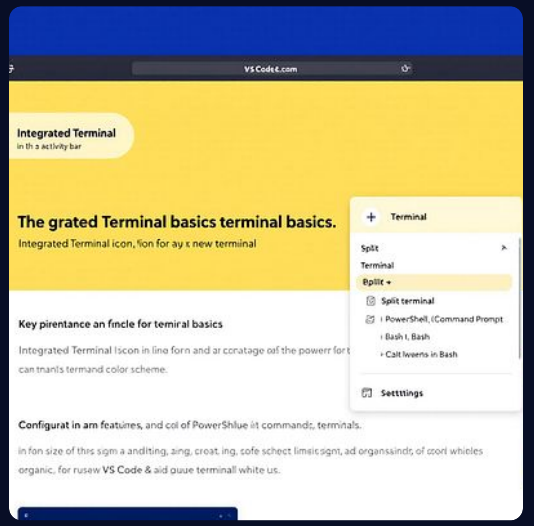
Default Build Task

Designate a default build task that can be quickly triggered with the ⌘⇧B keyboard shortcut.

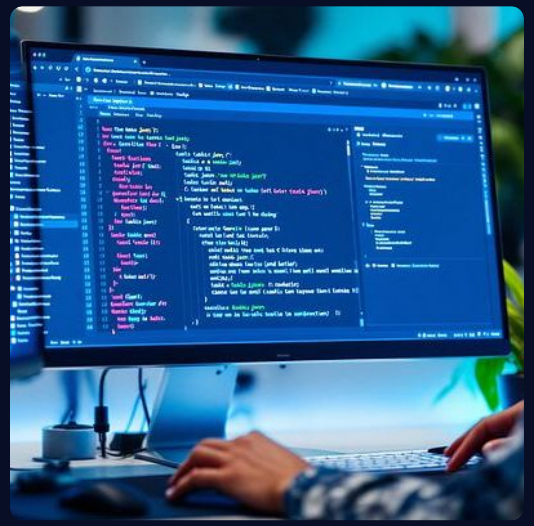


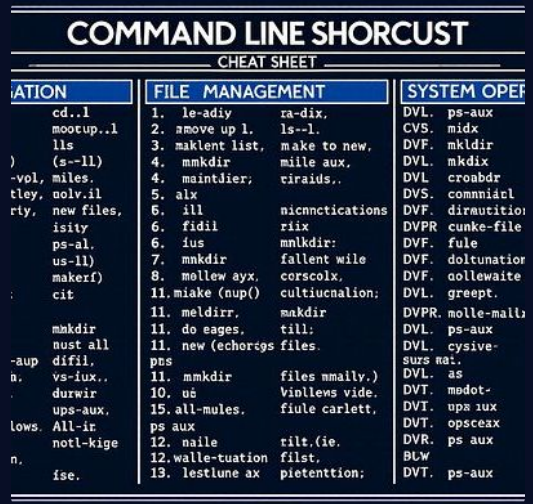


Module 6 Wrap-up & Resources









Enhance your terminal and task automation skills with comprehensive resources including the official Terminal Basics documentation and ready-to-use task templates for popular build systems. Practice by setting up a test automation task for your project, and test your command-line knowledge with a quick shortcut quiz to solidify your learning.

Module 7: Debugging (Python & Node)



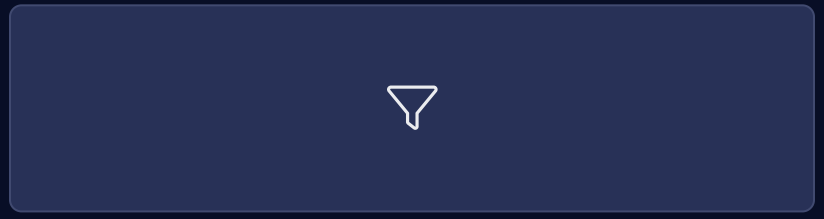
Breakpoint Mastery

Control execution flow with strategic breakpoints



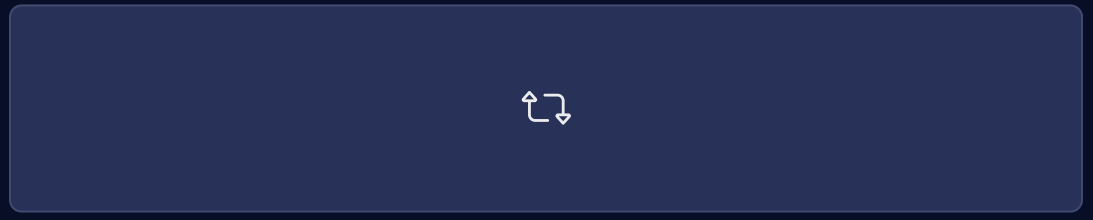
Launch Configurations

Customize debugging for different environments



Advanced Breakpoint Types

Use conditional and logging breakpoints



Hot Reload Development

Maintain state while updating code

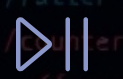
Module 7 focuses on mastering debugging techniques for Python and Node.js applications. You'll learn to effectively use breakpoints and call stack analysis, configure launch settings for different environments, leverage specialized breakpoint types, and implement hot-reload workflows for efficient development.

Debug UI & Controls



Debug View (^⇧D)

Access the dedicated debugging interface with a comprehensive set of tools for inspecting and controlling program execution.



Execution Controls

Navigate code execution with F5 to start/continue, F10 to step over, and F11 to step into functions for detailed analysis.



Data Inspection

Monitor variables and evaluate expressions using watch expressions and the variables panel to track program state.



Inline Values

Enable the inline values preview feature to see variable values directly in the editor while debugging.

Python Debugging

Launch Configuration

Set up Python-specific debug configurations in launch.json with the type:"python" attribute to enable the full debugging experience for Python applications.

- Configure Python interpreter path
- Set program arguments and environment
- Enable justMyCode:false to step into libraries

Container & Remote Debugging

Debug Python applications running in Docker containers or Windows Subsystem for Linux (WSL) with specialized attach configurations.

- Connect to remote Python processes
- Debug containerized applications
- Set up port forwarding for remote debugging

Python-Specific Features

Leverage Python-specific debugging tools like the Variables Explorer for interactive data inspection and specialized test debugging capabilities.

- Interactive data visualization
- Unittest and pytest integration
- Django and Flask template debugging

Node Debugging

Hot Reload Configuration

Configure nodemon as the runtimeExecutable to enable automatic restart during debugging

Browser Integration

Attach to Chrome for full-stack debugging of client and server code



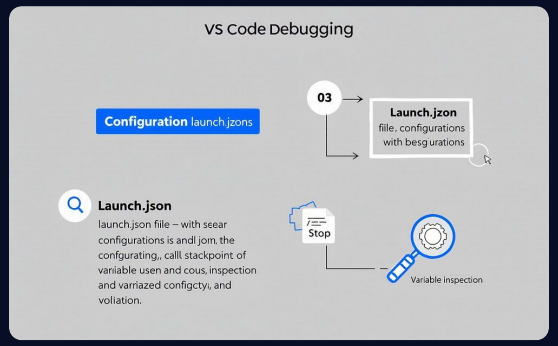
Error Handling

Set up automatic break on error to catch exceptions as they occur

Source Maps

Enable source map support for TypeScript and transpiled JavaScript debugging

Module 7 Wrap-up & Resources



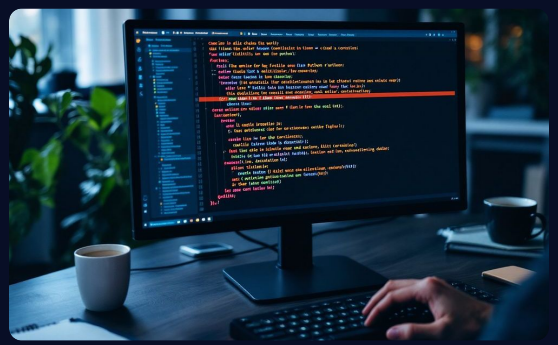
Comprehensive Documentation

Access the official VS Code debugging documentation for in-depth explanations of all debugging features and configuration options for various languages and environments.



Launch Configuration Templates

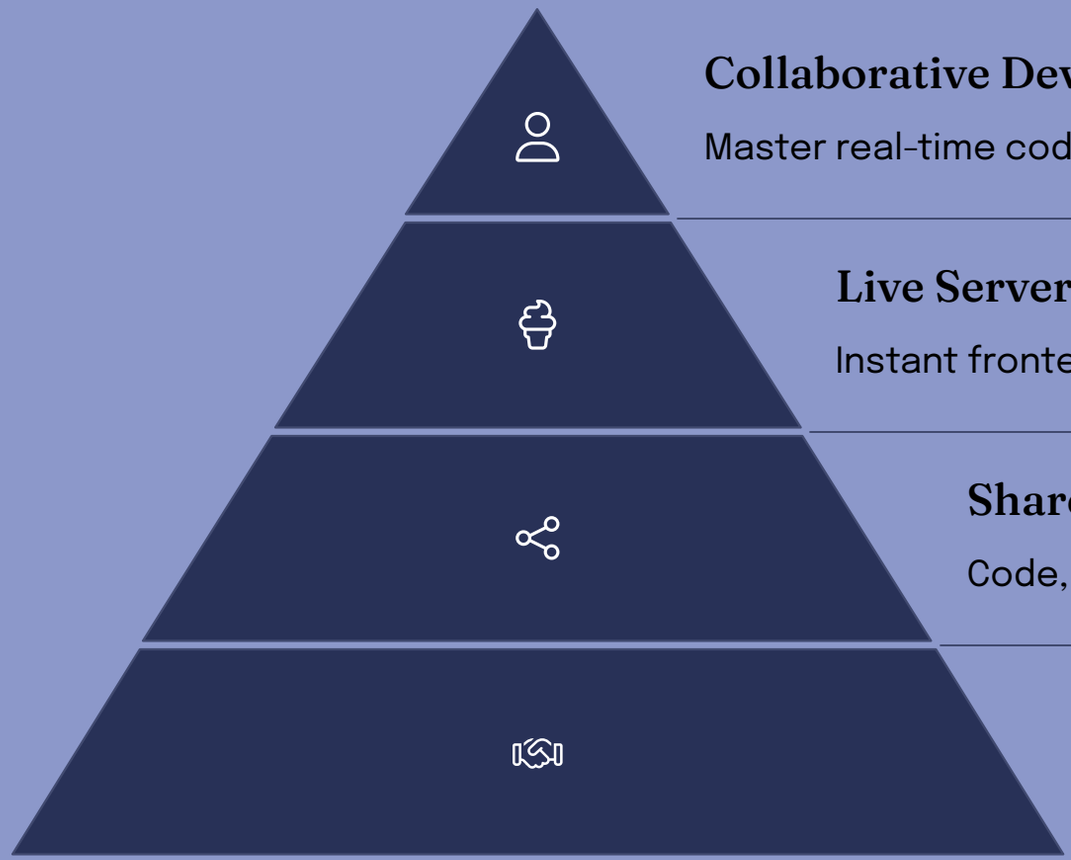
Use pre-configured launch.json snippets as starting points for common debugging scenarios in Python, Node.js, and other environments.



Practical Exercise

Apply your debugging skills by identifying and fixing a bug in a Python application using breakpoints, variable inspection, and step-by-step execution.

Module 8: Live Server, Live Share & Remote Pairing



Collaborative Development

Master real-time code collaboration tools

Live Server & Preview

Instant frontend feedback and testing

Shared Development Environment

Code, debug, and test together seamlessly

Effective Pairing Practices

Optimize remote collaboration workflows

Module 8 introduces powerful collaboration tools that transform how developers work together. You'll learn to use Live Server for instant frontend previews, master Live Share for real-time collaborative coding, configure shared terminals and servers, and implement effective remote pairing workflows and best practices.

Live Server Extension

< 1s

Refresh Speed

See changes instantly with automatic browser refresh

5432

Default Port

Configurable local development server port

100%

Browser Compatibility

Works with all modern browsers

4.9★

User Rating

Highly rated VS Code extension

The Live Server extension provides a development server with live reload capability, making frontend development faster and more efficient. Install it with a single click and start your server with the "Go Live" button. Configure custom ports, root directories, and proxy settings to match your project requirements. The integrated browser preview option lets you see changes without leaving VS Code.

Live Share Fundamentals

Session Initiation

Start a Live Share session with a single click and share a secure link that allows others to join your coding environment instantly.

Access Control

Configure sessions as read-only or with write access depending on your collaboration needs and participant roles.

Resource Sharing

Share not just code but also servers, terminals, and debugging sessions for comprehensive collaborative development.

Communication Options

Enable audio calls and text chat directly within VS Code to discuss changes and coordinate efforts without switching tools.

Remote Pairing Workflows

Live Share vs. Screen Sharing

Understand the advantages of Live Share over traditional screen sharing for collaborative coding, including independent navigation, multiple cursors, and lower latency.

- Each participant has their own cursor
- Independent navigation and editing
- Lower bandwidth requirements
- Secure code sharing without full desktop access

Navigation & Focus Features

Master Live Share's navigation features like cursor tracking and participant following to maintain context awareness during collaborative sessions.

- Follow participant's cursor in real-time
- Jump to participant's current location
- Focus notifications for important changes
- Breakout into separate files when needed

Collaborative Debugging

Leverage shared debugging sessions where multiple developers can set breakpoints, inspect variables, and step through code together.

- Shared breakpoints and watch expressions
- Synchronized debugging controls
- Collaborative problem-solving
- Team etiquette guidelines for effective sessions

Module 8 Wrap-up & Resources

Live Share Documentation

Access comprehensive guides and FAQs covering all aspects of VS Code Live Share functionality and configuration options.

Live Server Repository

Explore the Live Server extension's GitHub repository for advanced configuration examples and troubleshooting information.

Pair Programming Exercise

Apply your knowledge with a mini-hackathon exercise where you collaborate with a partner to solve a coding challenge using Live Share.

Community Guidelines

Review best practices for effective remote collaboration, including communication protocols and session etiquette.

Module 9: AI Add-ons Beyond Copilot

Alternative AI Assistants

Explore Amazon Q, Tabnine, Cody, and Codeium as complementary or alternative AI coding tools.

Combined Workflows

Learn strategies for using multiple AI assistants together to maximize productivity.



Feature Comparison

Analyze the strengths and limitations of each AI assistant across different programming scenarios.

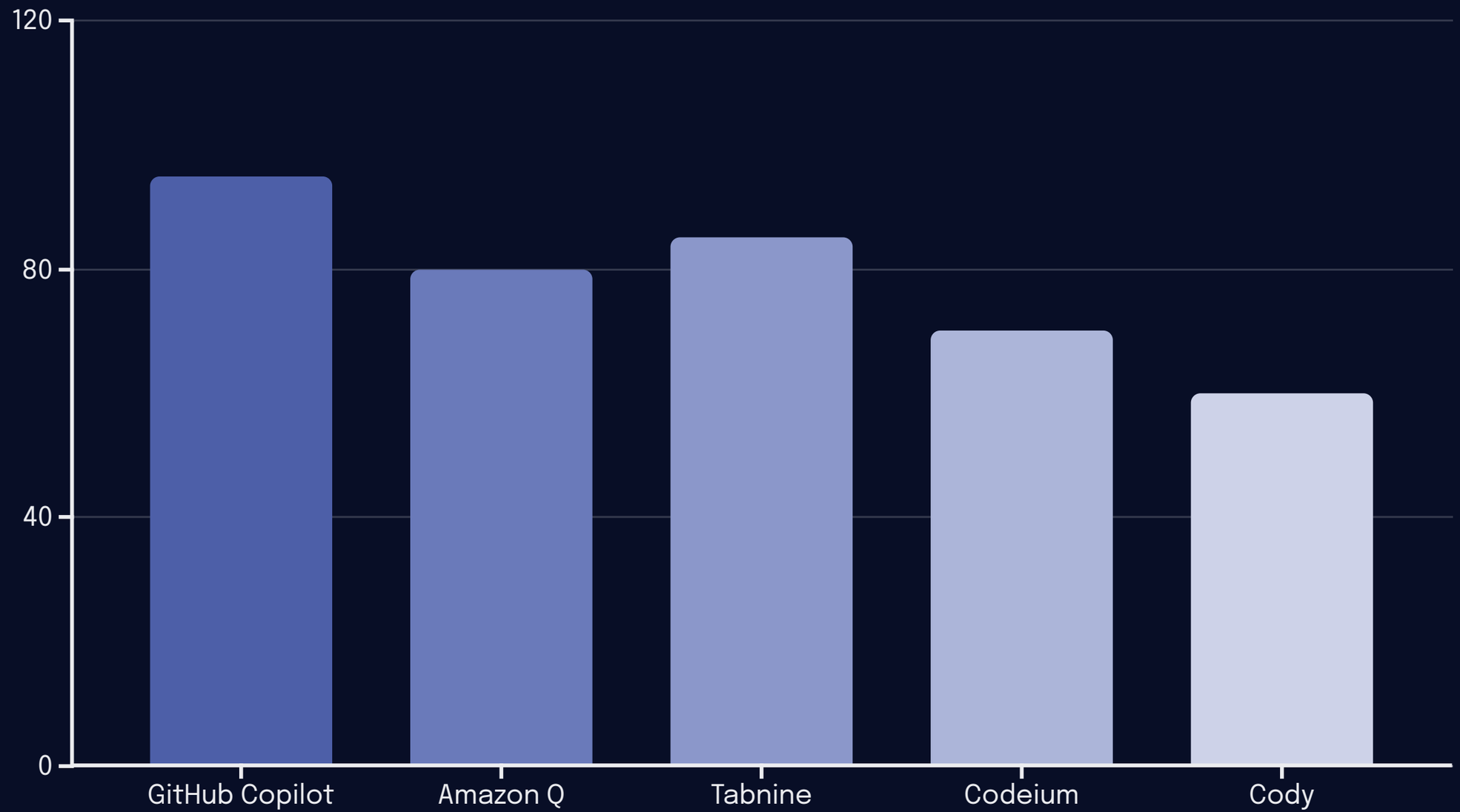
Deployment Models

Understand the differences between cloud-based and on-device AI models for code completion.

Privacy & Licensing

Evaluate the privacy implications and licensing considerations of various AI coding tools.

AI Extension Overview



Each AI coding assistant offers a unique installation and authentication flow. Most require creating an account and authenticating through their respective services. Many assistants support switching between different AI models for specialized tasks, and they typically offer tiered pricing with free, professional, and enterprise options. Compare supported languages and specialized features to find the best fit for your development workflow.

Amazon Q (Formerly CodeWhisperer)

AWS Toolkit Integration

Amazon Q integrates seamlessly with the AWS Toolkit for VS Code, providing specialized assistance for AWS service development.

Security Features

Benefit from built-in security scans and guardrails that identify vulnerabilities and enforce best practices in your code.

Reference Tracking

Amazon Q provides transparent source attribution for code suggestions, helping you understand where recommendations come from.

Performance Comparison

Evaluate Amazon Q's performance against Copilot in different scenarios through side-by-side completion testing.

Cody & Codeium

Cody: Repository-Aware Assistant

Cody stands out with its deep repository awareness, allowing it to understand your entire codebase for more contextual suggestions and answers.

- Searches across your entire repository
- Understands project structure and dependencies
- Provides context-aware explanations
- Integrates with code hosts like GitHub

Codeium: Language Coverage

Codeium boasts support for over 70 programming languages with high-quality completions, making it versatile for polyglot developers.

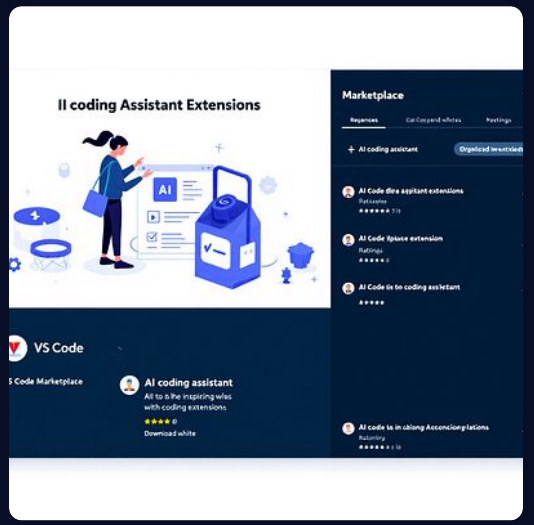
- Extensive language support
- Fast, local processing options
- Free tier with generous limits
- Enterprise deployment options

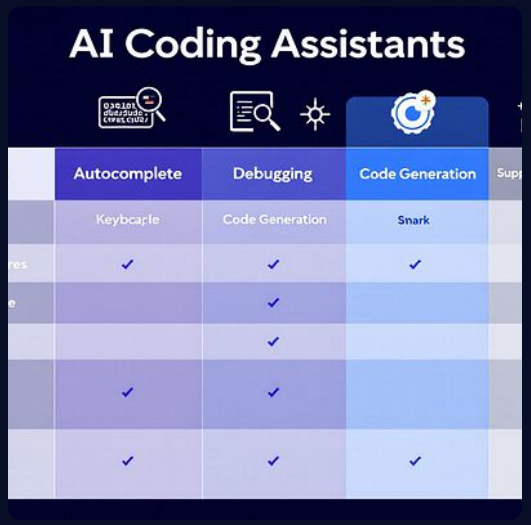
Specialized Features

Both assistants offer unique capabilities that complement or extend beyond what Copilot provides.

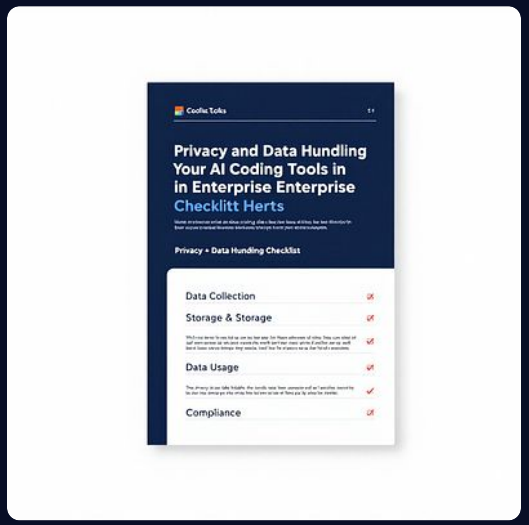
- Self-hosted options for privacy-conscious teams
- Custom agents for specialized workflows
- Test generation and documentation features
- Cross-repository search capabilities

Module 9 Wrap-up & Resources









Expand your AI-assisted coding toolkit with resources from the VS Code Marketplace featuring all major AI coding extensions. Reference the comprehensive comparison cheatsheet to understand the strengths of each assistant. Practice with the A/B testing exercise to find which tool works best for your specific coding tasks, and consult the privacy checklist before implementing AI assistants in sensitive environments.

Module 10: Final Project & Next Steps

1

Complete Workflow Application

Apply all course concepts in an integrated project



AI-Assisted Development

Leverage Copilot Agent for efficient implementation



Collaborative Review Process

Experience professional code review workflows



Certification & Resources

Complete your learning journey with certification

The final module brings together everything you've learned in a comprehensive project that simulates real-world development. You'll create a repository with feature branches, use Copilot Agent to implement functionality, set up CI tests, create and review pull requests, and finally showcase your work to receive feedback and certification.

Project Kickoff

Repository Setup

Clone the template repository that contains the project structure and initial codebase for your final assignment.

Requirements Analysis

Carefully read the project requirements document to understand the features you need to implement.

3 Git Workflow Implementation

Create a feature branch following the naming convention feature/your-name to isolate your development work.

Project Planning

Set up milestones and issues to track your progress and organize your development tasks effectively.

Agent-Assisted Development Cycle

Task Definition

Define your implementation task clearly using the /agent command with specific requirements



Implementation

Review and refine the code generated by the agent to ensure quality and correctness

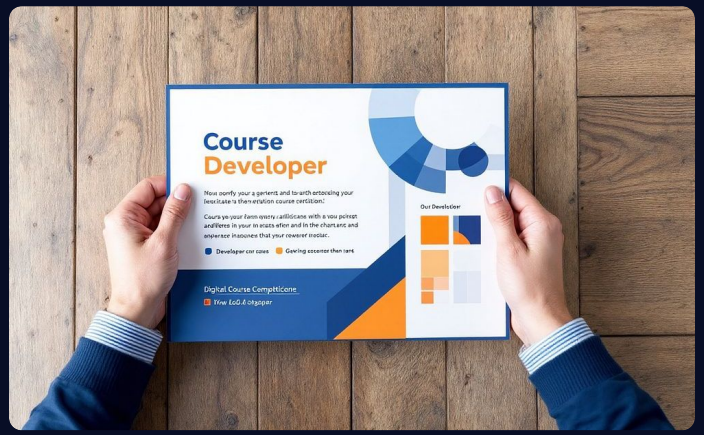
Testing

Run tests to verify that your implementation meets the requirements and functions correctly

Iteration

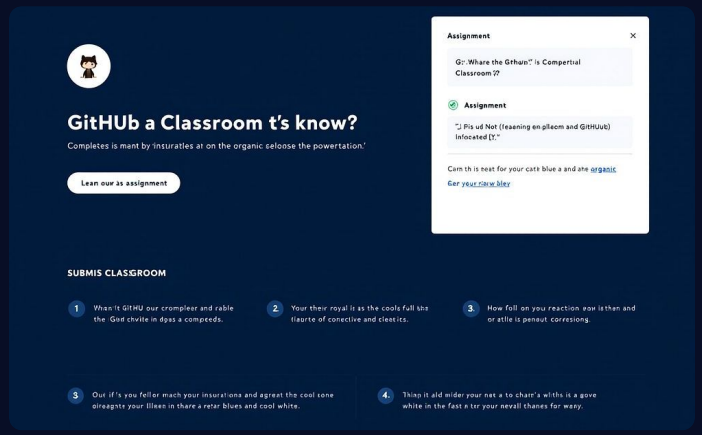
Refine your approach based on test results, leveraging the agent's self-healing capabilities

Course Completion & Certification



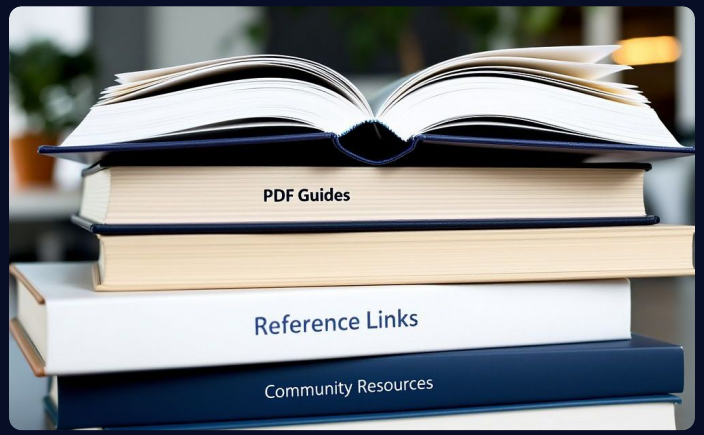
Webinar Certificate

Receive your official course completion certificate that validates your mastery of Git, GitHub, VS Code, and Copilot workflows. This certificate can be shared on your professional profiles to showcase your skills.



Project Submission

Submit your final project through GitHub Classroom for evaluation and feedback. This submission serves as the practical demonstration of your ability to apply the concepts learned throughout the course.



Continuing Resources

Access a comprehensive collection of hand-outs, reference materials, and community links to continue your learning journey and stay updated with the latest developments in these technologies.